PlanetaFP

Curso de Especialización en
Inteligencia Artificial y Big Data
MODALIDAD ONLINE

DOCUMENTACIÓN Y DISEÑO TÉCNICO

Predicción de demanda y precio de la electricidad en España

Sistema basado en datos abiertos de Red Eléctrica (REData / ESIOS) y meteorología, con API de predicción y dashboard.



INTEGRANTES DEL EQUIPO

David Lopez Solera | Kevin Tortosa Villanueva | Pablo Fernandez Fernandez | Juan Carlos Castro Pazo

DATOS + IA = UN FUTURO MÁS EFICIENTE

ÍNDICE

1. Título del proyecto e integrantes.
2. Resumen ejecutivo.
3. Descripción funcional del sistema.
4. Alcance definitivo (IN / OUT).
5. Arquitectura del sistema (alto nivel).
6. Tecnologías seleccionadas y justificación.
7. Modelo de datos.
8. Datos: Calidad, Riesgos y Decisiones.
9. Ética, legalidad y accesibilidad.
10. Backlog inicial de historias de usuario.
11. Plan de trabajo (Cronograma Semanal).
12. Estrategia de desarrollo y MLOps.
13. Riesgos técnicos actualizados y mitigación.
14. Repositorio del proyecto.

Documento de Diseño Técnico (NF2)

1. Título del proyecto e integrantes

- **Nombre del proyecto:** Predicción de demanda y precio de la electricidad en España.

- **Integrantes y roles:**

    - **David López Solera** - *Data Engineer*: Responsable de la ingesta de las tres fuentes principales (REData, ESIOS, Open-Meteo), persistencia en local, limpieza de anomalías, imputación de nulos, alineación de series temporales a UTC y diseño de la arquitectura de almacenamiento en Parquet.
    - **Kevin Tortosa Villanueva** - *Platform/MLOps y coordinación*: Responsable de la infraestructura de contenedores (Docker y docker-compose.yml), integración continua mediante GitHub Actions, inyección de secretos/variables de entorno, revisión de políticas de repositorio, tablero Kanban y facilitación de las reuniones semanales.
    - **Pablo Fernández Fernández** - *ML Engineer*: Responsable del Análisis Exploratorio de Datos (EDA), ingeniería de características (lags, variables de calendario), definición y evaluación de modelos Baseline, y entrenamiento/optimización de modelos de Gradient Boosting (LightGBM/XGBoost).
    - **Juan Carlos Castro Pazo** - *API & Dashboard*: Responsable del desarrollo del backend sirviendo predicciones vía FastAPI, diseño de contratos JSON, y construcción de la interfaz gráfica interactiva utilizando Streamlit y Plotly.

2. Resumen ejecutivo

La electricidad no puede almacenarse a gran escala; su generación debe igualar a la demanda en cada instante. En consecuencia, el precio del mercado diario varía fuertemente. Anticipar esto es crítico para el operador (REE), comercializadoras y consumidores (PVPC).

Este proyecto desarrolla un sistema de Machine Learning para predecir la demanda eléctrica horaria en la España peninsular a 24 horas (horizonte D+1) y, como objetivo secundario, el precio del mercado diario.

La solución automatiza la ingesta de datos abiertos (REData, ESIOS, Open-Meteo) mediante un pipeline reproducible. Aporta valor al unificar fuentes dispersas y superar los modelos heurísticos de referencia (baselines) usando algoritmos de Gradient Boosting. Todo el sistema está containerizado con Docker y expone las predicciones de forma sencilla a través de una API REST asíncrona y un dashboard interactivo.

3. Descripción funcional del sistema

3.1. Funcionalidad central

El sistema actúa como un orquestador de datos y predicciones. Diariamente, automatiza la descarga de series históricas actualizadas (demanda, precio y clima), transforma estos datos en bruto en variables predictivas (features) y alimenta un modelo preentrenado. Finalmente, los resultados son servidos para su explotación analítica.

3.2. Perfiles de usuario y flujos de interacción

El sistema está diseñado para atender a dos tipos de consumidores finales:

- **Usuario Analista/Negocio:** Accede al Dashboard (Streamlit). Su flujo de uso consiste en abrir la aplicación web, donde visualizará de inmediato un gráfico comparativo de la "Demanda Real vs. Predicción" del día en curso. Puede utilizar selectores de fecha para analizar el rendimiento del modelo en días pasados o visualizar las proyecciones de las próximas 24 horas, apoyándose en métricas de error (MAE, MAPE) calculadas en tiempo real para generar confianza en la inferencia.
- **Usuario Técnico/Desarrollador:** Accede a la API REST (FastAPI). Su flujo consiste en realizar peticiones HTTP GET o POST al endpoint /predict. El sistema le devuelve un payload estructurado en JSON con el array de predicciones horarias, listo para ser consumido por otras aplicaciones de terceros, sistemas de domótica o bots automatizados.

3.3. Ciclo de vida del dato (Paso a paso)

1. **Extracción:** Un script programado solicita el último bloque de datos a las APIs de REE y meteorología.
2. **Carga en crudo (Raw):** Los JSONs de respuesta se almacenan sin alterar para mantener linaje de datos.
3. **Transformación (Processed):** Un pipeline de limpieza elimina duplicados, interpola huecos menores, normaliza los timestamps a UTC (resolviendo los días de 23/25h por cambios de hora) y los guarda en archivos .parquet.
4. **Inferencia:** El módulo de ML carga el último modelo .pkl, genera las predicciones sobre la ventana de 24 horas solicitada y guarda el resultado.
5. **Servicio:** La API expone estas predicciones almacenadas a la capa de frontend.

3.4. Límites del sistema (Qué NO hace)

- No realiza reentrenamiento automático en línea (online learning); los modelos se entrenan estáticamente y se actualizan por lotes (batch).
- No predice fluctuaciones intradiarias bruscas ni mercados de ajuste secundarios.
- No desglosa la predicción por nodos, provincias ni comunidades autónomas (la predicción es agregada a nivel Península).

4. Alcance definitivo (IN / OUT)

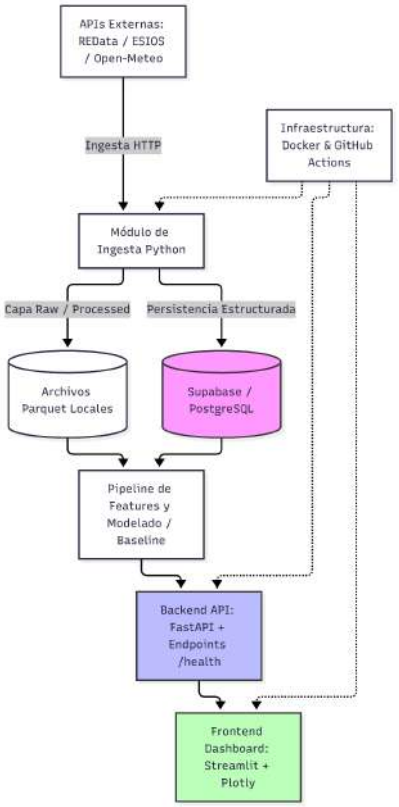
Elementos Incluidos (IN - Funcionalidades Obligatorias y Core)

- Modelado y predicción de la demanda horaria de España peninsular a 24 horas.
- Extracción y almacenamiento de histórico plurianual (2019-2025).
- Inclusión de variables meteorológicas (temperatura) de 4-5 ciudades representativas de la península.
- Desarrollo de un pipeline completo de ingesta y limpieza en Python (Pandas/Requests).
- Entrenamiento y validación Walk-Forward de modelos de la familia Gradient Boosting (LightGBM o XGBoost).
- Construcción de una API REST documentada (Swagger/OpenAPI) mediante FastAPI.
- Construcción de un Dashboard exploratorio mediante Streamlit y Plotly.
- Despliegue de la solución utilizando contenedores Docker (docker-compose).
- Estimación del precio del mercado diario (definido explícitamente como objetivo secundario).

Elementos Excluidos (OUT - Descartados Explícitamente)

- Predicción en tiempo real (streaming con Kafka o Spark Streaming).
- Uso de arquitecturas complejas de Deep Learning (Redes Neuronales, LSTM, Transformers), debido a la relación coste-beneficio computacional frente a ensambles de árboles.
- Desglose geográfico o por tipo de consumidor.
- Recomendaciones financieras, estrategias de trading o compra-venta de energía.
- Sistemas de autenticación, gestión de usuarios (JWT/OAuth) o roles dentro de la API/Dashboard.
- Despliegue en clusters de Kubernetes o Cloud Services de pago.

5. Arquitectura del sistema (alto nivel)



La solución sigue una arquitectura orientada a microservicios monolíticos paquetizados en contenedores, dividida en tres grandes bloques lógicos:

6. **Bloque de Datos (Data Pipeline):**
    - **Extractors:** Scripts modulares que atacan las APIs REST externas (REData, ESIOS, Open-Meteo). Gestionan los retries, timeouts y la paginación.
    - **Storage Layer (Data Lake local):** Sistema de archivos particionado localmente, montado como volúmenes en Docker. Dividido en data/raw/ (JSON crudo inmutable) y data/processed/ (archivos Parquet optimizados con diccionarios de compresión Snappy).

7. **Bloque de Machine Learning (ML Pipeline):**
    - **Feature Store (Lógica):** Scripts de transformación que generan variables cíclicas (seno/coseno de la hora y mes), lags (demanda hace 24h, hace 168h), medias móviles y variables booleanas (es_fin_de_semana, es_festivo).
    - **Training Engine:** Módulo offline ejecutado esporádicamente que particiona los datos, realiza validación cruzada temporal (Walk-Forward) y serializa el artefacto ganador.

8. **Bloque de Servicio (Serving Layer):**
    - **Backend Container (FastAPI):** Expone la lógica de negocio. Carga el artefacto del modelo en memoria RAM al arrancar con Uvicorn para minimizar la latencia de inferencia. Expone endpoints /api/v1/predict y /api/v1/health.
    - **Frontend Container (Streamlit):** Servicio web independiente que actúa como cliente de la API interna. Renderiza los gráficos interactivos sin contener lógica pesada de procesamiento de datos.

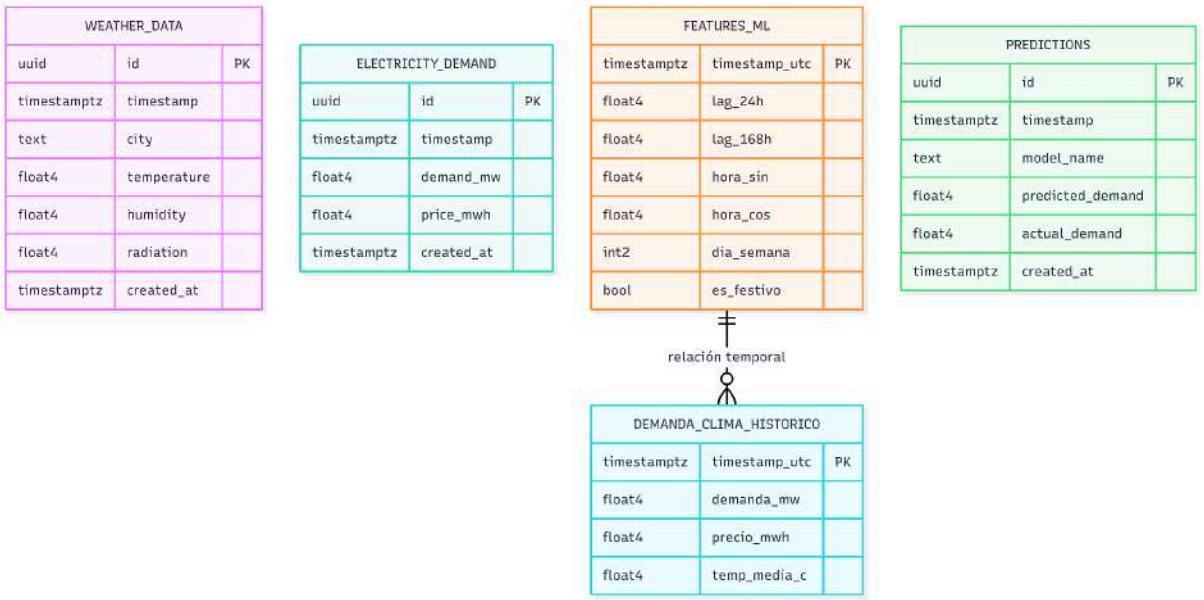
Todos los contenedores se orquestan bajo una red privada de puente (bridge network) en Docker Compose, impidiendo el acceso directo externo a los datos y exponiendo únicamente los puertos necesarios para la API y el Dashboard.

6. Tecnologías seleccionadas y justificación

- **Lenguaje Base:** Python 3.11+. Se elige por ser el estándar de la industria en datos, ofreciendo soporte nativo desde la ingesta (Requests) hasta el modelado (Scikit-Learn).
- **Manipulación de Datos:** Pandas y PyArrow. Pandas es óptimo para manipulación tabular en memoria y alineación de series temporales. PyArrow es el motor subyacente elegido para leer/escribir archivos Parquet a máxima velocidad.
- **Almacenamiento Físico:** Formato Apache Parquet. Justificación: Reduce el peso del histórico en un 70-80% frente a CSV, conserva los tipos de datos nativos de las columnas y permite cargas parciales rápidas. (Se descarta PostgreSQL por introducir complejidad infraestructural innecesaria para series temporales de este volumen).
- **Machine Learning:** LightGBM (Microsoft) o XGBoost. Se eligen frente a ARIMA o Redes Neuronales porque manejan nativamente relaciones no lineales, ignoran características irrelevantes y son computacionalmente ligeros para entrenar y predecir.
- **Capa Backend:** FastAPI. Elegido por su velocidad (basado en Starlette y Pydantic), tipado estático nativo y generación automática de la documentación de la API (Swagger UI).
- **Capa Frontend:** Streamlit + Plotly. Elegido porque permite a los científicos de datos construir aplicaciones interactivas puramente en Python sin necesidad de escribir HTML, CSS o JavaScript.

- **Infraestructura y DevOps:** Docker, Docker Compose y GitHub Actions. Permite aislar dependencias del sistema operativo del host y automatizar pruebas.

7. Modelo de datos



7.1. Entidad: weather_data **(Datos Climáticos Crudos)**
Almacena las observaciones meteorológicas desagregadas por ciudades de referencia, utilizadas como variables exógenas en los modelos predictivos.

- id (uuid): Identificador único universal del registro.
- id (id, PK): Identificador autoincremental de control.
- timestamp (timestamptz): Marca temporal horaria de la observación climática.
- city (text): Nombre de la ciudad de referencia meteorológica.
- temperature (float4): Temperatura ambiental registrada en grados Celsius (°C).
- humidity (float4): Humedad relativa del aire expresada en porcentaje (%).

- radiation (float4): Nivel de radiación solar incidente.

- created_at (timestamptz): Fecha y hora exacta de inserción o registro en la base de datos.

7.2. Entidad: electricity_demand **(Demanda y Precios Históricos Crudos)**
Contiene la serie temporal oficial de la demanda eléctrica real registrada en el sistema peninsular y los precios asociados del mercado diario.

- id (uuid): Identificador único universal del registro.

- id (id, PK): Identificador autoincremental de control.

- timestamp (timestamptz): Marca temporal horaria correspondiente a la medida.

- demand_mw (float4): Demanda real de energía eléctrica medida en Megavatios (MW).

- price_mwh (float4): Precio del mercado diario en euros por Megavatio-hora (€/MWh).

- created_at (timestamptz): Fecha y hora exacta de inserción del registro.

7.3. Entidad: features_ml **(Ingeniería de Características)**
Almacena las variables transformadas y calculadas a partir del preprocesamiento, optimizadas para alimentar directamente el pipeline de entrenamiento y los algoritmos de Machine Learning.

- timestamp_utc (timestamptz, PK): Marca temporal estandarizada en UTC que actúa como clave primaria de la tabla.

- lag_24h (float4): Retardo temporal de la demanda correspondiente a exactamente 24 horas antes.

- lag_168h (float4): Retardo temporal de la demanda correspondiente a una semana antes (168 horas).

- hora_sin (float4): Componente trigonométrica sinusoidal para la codificación cíclica de la hora del día.

- hora_cos (float4): Componente trigonométrica cosenoidal para la codificación cíclica de la hora del día.

- dia_semana (int2): Valor numérico entero representativo del día de la semana.

- es_festivo (bool): Indicador booleano que señala si el día es festivo nacional.

7.4. Entidad: demanda_clima_historico **(Dataset Consolidado)**

Tabla consolidada que vincula las características temporales con los valores agregados de demanda, precio y temperatura media de la península para la fase analítica y de modelado.

- timestamp_utc (timestamptz, PK): Marca temporal estandarizada en UTC, relacionada directamente con la tabla de características.

- demanda_mw (float4): Demanda real consolidada en Megavatios (MW).

- precio_mwh (float4): Precio consolidado del mercado diario en €/MWh.

- temp_media_c (float4): Temperatura media ponderada de las ciudades de referencia en grados Celsius (°C).

7.5. Entidad: predictions **(Predicciones y Auditoría de Modelos)**

Tabla dedicada a almacenar las inferencias generadas por la API de los modelos y contrastarlas frente a la realidad para la evaluación continua del rendimiento.

- id (uuid): Identificador único universal de la predicción.

- id (id, PK): Identificador autoincremental de control.

- timestamp (timestamptz): Marca temporal horaria para la cual se calcula la predicción.

- model_name (text): Nombre e identificador del modelo ejecutor (ej. *baseline*, *lightgbm_v1*).

- predicted_demand (float4): Valor de demanda estimada por el modelo en Megavatios (MW).

- actual_demand (float4): Demanda real contrastada posteriormente para el cálculo de errores (MAE/MAPE).

- created_at (timestamptz): Momento exacto en que se ejecutó la inferencia (timestamp de inferencia).

8. Datos: Calidad, Riesgos y Decisiones

La materia prima del proyecto consiste en series temporales numéricas. En total se procesarán en torno a 25.000 registros horarios por cada métrica, abarcando 3 años (2023-2025).

- **Fuentes:** REData (API abierta), ESIOS (API restringida, requiere Token), Open-Meteo (API abierta).
- **Calidad y Riesgos:** Las series eléctricas suelen ser consistentes, pero presentan problemas críticos durante los cambios de horario estacionales (un día en marzo tiene 23 horas, un día en octubre tiene 25 horas).
- **Naturaleza:** Los datos son 100% reales, públicos e históricos. No se utilizarán datos sintéticos ni mockeados.

Tabla de decisiones de datos y supuestos de modelado

- **ESIOS (Precio):** Se asume como un objetivo secundario con alta volatilidad. Decisión: Si el modelo de regresión no logra superar al baseline, se limitará el alcance a predecir por 'franjas' (cara/media/barata) en lugar de buscar un valor numérico exacto.

- **REData (Demanda)**: Se asume como la variable objetivo principal (ground truth). Decisión: Alinear y normalizar el histórico a UTC para evitar desfases con los días de 23h y 25h en los cambios de hora estacionales.

- **Open-Meteo / AEMET:** Se asume que la temperatura de un pequeño grupo de ciudades representativas es suficiente para correlacionar con la demanda peninsular total. Decisión: Se asume y acepta el riesgo de desviación natural entre la previsión meteorológica y el dato real observado.

9. Ética, legalidad y accesibilidad

- **Privacidad y GDPR:** El proyecto utiliza exclusivamente datos macroeconómicos, climáticos y de redes de transporte de energía. Existe una ausencia total de Información de Identificación Personal (PII).
- **Legalidad de Fuentes:** El consumo de ESIOS se realiza acatando su política de uso justo mediante la obtención de un token personal individual.
- **Seguridad de Secretos:** Los tokens de API nunca se versionarán en Git. Se gestionarán exclusivamente mediante archivos .env locales.
- **Accesibilidad y Sesgos:** Al no operar sobre decisiones humanas (créditos, recursos humanos, justicia), el modelo carece de sesgo demográfico o social. El dashboard se configurará utilizando temas de alto contraste.

10. Backlog inicial de historias de usuario

- **HU01:** Como DevOps, quiero configurar la infraestructura base en Docker para que todo el equipo comparta el mismo entorno.
    - *Criterio de aceptación:* `docker-compose up` levanta todos los contenedores y servicios sin errores de dependencia ni conflictos de puertos.

- **HU02:** Como Data Engineer, quiero un script automático que descargue el histórico de REData para asegurar una fuente fiable local.
    - *Criterio de aceptación:* El script descarga correctamente las series temporales y guarda los datos originales en formato JSON dentro de `data/raw/`.
- **HU03:** Como Data Engineer, quiero ingerir datos climáticos de Open-Meteo para tener variables exógenas del clima.
    - *Criterio de aceptación:* Se obtienen las variables horarias de temperatura, humedad y radiación para las coordenadas clave, almacenándose en el directorio correspondiente sin valores nulos críticos.
- **HU04:** Como Data Engineer, quiero limpiar y transformar los datos brutos a Parquet UTC para alimentar los modelos eficientemente.
    - *Criterio de aceptación:* Los ficheros resultantes se generan en formato `.parquet` normalizados a UTC y comprimidos en Snappy.
- **HU05:** Como ML Engineer, quiero realizar el Análisis Exploratorio (EDA) para comprender la estacionalidad de la demanda.
    - *Criterio de aceptación:* Generación de un informe Jupyter Notebook visualizando autocorrelaciones horarias, diarias y semanales de la demanda.
- **HU06:** Como ML Engineer, quiero crear un modelo Baseline Naive (24h/168h lag) para tener una línea base estricta de evaluación.
    - *Criterio de aceptación:* Se calcula y registra el MAE y MAPE del baseline sobre el conjunto de validación temporal.
- **HU07:** Como ML Engineer, quiero generar variables de calendario (festivos, trigonométricas) para mejorar la precisión algorítmica.
    - *Criterio de aceptación:* Inclusión exitosa de variables `hora_sin`, `hora_cos` y flags de festivos nacionales en el dataset de entrenamiento.
- **HU08:** Como ML Engineer, quiero entrenar y afinar un modelo Gradient Boosting para superar el umbral de precisión del Baseline.
    - *Criterio de aceptación:* El MAE del modelo entrenado mejora en al menos un 10% frente al Baseline en validación Walk-Forward.
- **HU09:** Como Backend Developer, quiero exponer un endpoint `/predict` en FastAPI para que la inferencia sea accesible externamente.
    - *Criterio de aceptación:* Peticiones POST con payload JSON devuelven las predicciones horarias estructuradas y la documentación Swagger se genera automáticamente en `/docs`.

- **HU10:** Como Frontend Developer, quiero un panel en Streamlit para visualizar interactivamente el histórico y las previsiones.
    - *Criterio de aceptación:* La interfaz web muestra gráficos interactivos de Plotly comparando demanda real frente a predicción de forma fluida.

11. Plan de trabajo (Cronograma Semanal)

- **Fase 1: Infraestructura e Ingesta Cruda (Semanas 1 y 2).** Hito: Histórico crudo completo descargado y versionado en local.
- **Fase 2: Procesamiento y Exploración (Semanas 3 y 4).** Hito: Datos limpios preparados; baseline documentado.
- **Fase 3: Modelado Core - Demanda (Semanas 5, 6 y 7).** Hito: Modelo de demanda entrenado, evaluado y serializado superando el Baseline.
- **Fase 4: Modelado Secundario - Precio (Semanas 8 y 9).** Adaptación del pipeline de modelado a la curva de precios del mercado diario.
- **Fase 5: Servicio y Exposición (Semanas 10 y 11).** Hito: API y Dashboard completamente funcionales en local.
- **Fase 6: Consolidación (Semana 12).** Refinamiento del código, pruebas finales end-to-end de levantamiento del sistema y documentación.

12. Estrategia de desarrollo y MLOps

12.1. Gestión del Código Fuente (Git Flow simplificado)

Se ha configurado un repositorio en GitHub con políticas estrictas de colaboración:

- **Ramas principales:** Rama main (producción estable) y ramas feature/[nombre-tarea] para desarrollo.
- **Integración del Código:** Prohibido el push directo a main. Toda subida requiere Pull Request (PR).
- **Revisión por Pares (Code Review):** Reglas de protección de rama requieren aprobación de otro miembro.

12.2. Flujo de Integración Continua (CI)

- **Linting de código:** Ejecución de flake8 y black para asegurar convenciones de estilo PEP8.
- **Pruebas automáticas (Testing):** Ejecución de suite básica de pytest.
- **Validación de entorno:** Comprobación de que docker-compose build finaliza satisfactoriamente.

13. Riesgos técnicos actualizados y mitigación

- **Riesgo 1: Caída de la API de ESIOS o denegación por cuota.** Probabilidad: Media. Impacto: Alto. Mitigación: Mecanismo de almacenamiento local robusto. Plan B: Pivotar a históricos de ENTSO-E.
- **Riesgo 2: Dificultad para superar el modelo Baseline en precios.** Probabilidad: Alta. Impacto: Medio. Mitigación: Clasificar el precio explícitamente como objetivo secundario (franjas).
- **Riesgo 3: Expansión descontrolada del alcance (Scope Creep).** Probabilidad: Alta. Impacto: Crítico. Mitigación: Aplicación rigurosa de la metodología MoSCoW. Regla de recorte automático.
- **Riesgo 4: Problemas de rendimiento computacional en memoria local.** Probabilidad: Baja. Impacto: Medio. Mitigación: Uso exclusivo de formato Parquet.

14. Repositorio del proyecto

- **URL oficial:** https://github.com/Alive27/prediccion-electricidad
- **Estructura base del directorio:**
    - README.md, docker-compose.yml, .gitignore, .env.example
    - data/raw/
    - src/ingestion/, src/features/, src/models/, src/api/, src/dashboard/
    - Notebooks/, tests/, docs/